

Angular HandsOn 7: Routing and Navigation

HANDS-ON 7 [Intermediate]

Angular Routing ? Guards, Lazy Loading & Route Data

Topics Covered:

Configuring Routes & RouterModule[Check] Route Parameters & Query Parameters[Check]

Nested Routes[Check] Lazy Loading Feature Modules[Check]

CanActivate Guard[Check] CanDeactivate Guard[Check]

Passing Data Between Routes with Resolve[Check]

Routing is what makes Angular a true SPA framework. In this hands-on, you will configure all portal routes, add route parameters, implement lazy loading for the enrollment module, and protect routes with guards.

Task 1: Route Configuration, Parameters and Nested Routes

Goal: Set up all portal routes including dynamic segments and nested routes.

Steps:

68. In `app-routing.module.ts` (or `app.routes.ts` in standalone), define the following routes: `{ path: " } ?`

`HomeComponent, { path: 'courses' } ? CourseListComponent, { path: 'courses/:id' } ?`

`CourseDetailComponent` (generate this component), `{ path: 'profile' } ? StudentProfileComponent,`

`{ path: '**' } ?` a `NotFoundComponent` (generate it).

69. Generate `CourseDetailComponent`: `ng generate component pages/course-detail`. In it, inject

`ActivatedRoute` and read the `:id` parameter: `this.route.snapshot.paramMap.get('id')`. Use

`CourseService.getCourseById()` to load and display the matching course.

70. In `CourseListComponent`, make each course card clickable ? navigate to `/courses/:id` using

`Router.navigate(['courses', course.id])` when the card is clicked.

71. Add a query parameter to the courses route: when the search input has a value, update the URL to

`/courses?search=angular` using `this.router.navigate(['courses'], { queryParams: { search:`

`this.searchTerm } })`. Read it back in `ngOnInit` using `this.route.snapshot.queryParamMap.get('search')`.

72. Add nested routes under `/courses`: `{ path: 'courses', component: CoursesLayoutComponent, children:`

`[{ path: "", component: CourseListComponent }, { path: ':id', component: CourseDetailComponent } ] }`.

Add a `<router-outlet>` to `CoursesLayoutComponent`.

Hint:

- `route.snapshot.paramMap` is fine for parameters that do not change while the component is active. For parameters that can change (same component, different params), subscribe to `route.paramMap` `Observable` instead.

- The `**` wildcard route must always be the last route ? Angular matches routes in order, and a wildcard before specific routes would catch everything.

Expected Outcome: Navigating to `/courses/2` shows the correct course detail. Search updates the URL query param. The 404 page appears for unknown routes.

Angular HandsOn 7: Routing and Navigation

Task 2: Lazy Loading and Route Guards

Goal: Implement lazy loading for the enrollment feature and protect routes with guards.

Steps:

73. Create a feature module for enrollment: `ng generate module features/enrollment --routing`. Move `EnrollmentFormComponent` and `ReactiveEnrollmentFormComponent` into this module. Set up lazy loading in the main app routes: `{ path: 'enroll', loadChildren: () => import('./features/enrollment/enrollment.module').then(m => m.EnrollmentModule) }`.

74. Verify lazy loading works: open Chrome DevTools ? Network tab ? navigate to `/enroll` ? a separate JavaScript chunk file should be downloaded only when the route is first visited, not on initial app load.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book

75. Generate an auth guard: `ng generate guard guards/auth`. Implement `canActivate`: check if a property `isLoggedIn = true` (hardcoded for now) in an `AuthService`. Return true if logged in, otherwise navigate to `'/'` and return false.

76. Apply the guard to the `/profile` and `/enroll` routes: `{ path: 'profile', canActivate: [AuthGuard], component: StudentProfileComponent }`.

77. Generate a `CanDeactivate` guard for the reactive enrollment form: `ng generate guard guards/unsaved-changes`. Implement `canDeactivate` to check if the form is dirty (`enrollForm.dirty`). If dirty, use `window.confirm('You have unsaved changes. Leave?')` ? return true to allow navigation, false to stay.

Hint:

- Lazy loading splits the application into chunks that are downloaded on demand ? critical for large applications. Without it, all code is bundled into one large file downloaded on initial load.

- `CanDeactivate` guards prevent accidental loss of form data ? one of the most user-friendly features you can add to any form-heavy application.

Expected Outcome: Network tab shows a separate chunk loaded on first `/enroll` navigation. Navigating to `/profile` without login redirects to home. Dirty form shows a confirmation dialog on navigation away.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book